

Cours 08 — L'écosystème GitHub

Module 8/8 — Formation Git & GitHub

Objectifs

À la fin de ce module, vous saurez : - piloter GitHub depuis le terminal avec **GitHub CLI** (gh) — issues, PR, releases, workflows ; - publier un site statique avec **GitHub Pages** et organiser le travail avec **Projects** ; - sécuriser un dépôt : **Dependabot**, **secret scanning**, revue des permissions et protection de branche.

Théorie

GitHub CLI (gh) : GitHub sans quitter le terminal

gh apporte les objets GitHub — pull requests, issues, releases, workflows — dans votre terminal. Fini l'aller-retour permanent vers le navigateur pour les opérations courantes : on reste dans le flux de travail, à côté de git. Une seule authentification (gh auth login), puis les commandes reprennent la logique de l'interface web. C'est l'outil qui « recolle » tout ce que les cours précédents ont vu séparément.

GitHub Pages : héberger un site en un commit

GitHub Pages sert gratuitement des fichiers statiques (HTML, CSS, JS) depuis un dépôt. La source peut être une branche (main, dossier racine ou /docs) ou un artefact produit par un workflow Actions. L'URL suit le format `https://utilisateur.github.io/depot/`. C'est ce qui héberge la documentation d'un projet, un portfolio, une démo — sans serveur à gérer.

Projects : le kanban alimenté par le dépôt

Les **Projects** sont des tableaux (kanban, tableur, feuille de route) reliés automatiquement à vos **issues** et **PR**. Une issue déplacée dans la colonne « En cours », une PR fusionnée qui bascule sa carte en « Terminé » : le suivi de projet et le code vivent au même endroit, au lieu de se désynchroniser dans un outil séparé. On y ajoute des champs (priorité, sprint, estimation) et plusieurs vues sur les mêmes données.

Sécuriser un dépôt : quatre garde-fous

Outil	Rôle
Dependabot alerts	Signale les dépendances vulnérables (basé sur les CVE publiées)
Dependabot updates	Ouvre automatiquement des PR de mise à jour (dependabot.yml)
Secret scanning	Détecte les jetons/clés committés par erreur et alerte
Branch protection	PR + revue + CI obligatoires sur main (cours 04)

Ces quatre garde-fous se configurent **une fois** et travaillent ensuite en fond. Combinés à la CI du cours 06, ils forment une chaîne : Dependabot ouvre une PR de mise à jour, la CI vérifie qu'elle ne casse rien, la protection de branche empêche de fusionner si elle casse. La sécurité devient un processus, pas une corvée ponctuelle.

Commandes commentées

S'authentifier — gh auth login

```
gh auth login          # une seule fois par machine ; suivez
l'assistant
gh auth status         # vérifier à quel compte on est connecté
```

Assistant interactif : choisissez GitHub.com, le protocole (SSH recommandé si vous avez suivi le cours 03), et authentifiez-vous dans le navigateur. Ensuite, toutes les commandes gh sont autorisées.

Gérer les dépôts — gh repo

```
gh repo create mon-projet --public --clone # créer un dépôt sur GitHub ET le cloner en local
gh repo view --web # ouvrir le dépôt courant dans le navigateur
```

gh repo create remplace la séquence « aller sur le site → New repository → cloner » par une ligne. --public (ou --private) et --clone enchaînent tout.

Issues et PR — gh issue, gh pr

```
gh issue create --title "Bug export CSV" --body "Encodage incorrect"
gh issue list # les issues ouvertes du dépôt
gh pr create --fill # ouvrir une PR ; --fill reprend
titre/description des commits
gh pr checks # statut de la CI de la PR courante
gh pr merge --squash --delete-branch # fusionner en squash et supprimer la
branche
```

Tout le cycle du cours 04, sans navigateur. gh pr create --fill est un raccourci précieux : il remplit la PR à partir de vos messages de commit. gh pr merge --squash --delete-branch fusionne proprement et nettoie la branche dans la foulée.

Suivre les workflows — gh run

```
gh run list # dernières exécutions de workflow (statut,
branche, date)
gh run watch # suivre en direct l'exécution en cours
gh workflow run ci.yml # déclencher un workflow (workflow_dispatch)
```

Le pendant terminal de l'onglet **Actions** du cours 06 : lancer, lister, suivre les workflows sans lâcher le clavier.

Automatiser les mises à jour — .github/dependabot.yml

```
version: 2
updates:
  - package-ecosystem: pip
    directory: "/"
    schedule:
      interval: weekly
```

Ce fichier active **Dependabot updates**. Chaque semaine, Dependabot inspecte vos dépendances pip et ouvre **une PR par dépendance obsolète**. Votre CI du cours 06 valide automatiquement que la mise à jour ne casse rien — vous n'avez plus qu'à relire et fusionner.

Exercices pratiques

1. **CLI de bout en bout.** Installez gh, authentifiez-vous, puis réalisez un **cycle complet sans navigateur** : création d'issue, branche, commit, gh pr create, gh pr merge --squash.
2. **Pages.** Activez GitHub Pages sur un dépôt avec un index.html (ou index.md), et vérifiez l'URL publiée.
3. **Dependabot.** Ajoutez le dependabot.yml ci-dessus, épinglez volontairement une **vieille** version dans requirements.txt, et observez la PR de mise à jour automatique.
4. **Audit de sécurité.** Sur un de vos dépôts, passez en revue : protection de main, secret scanning, alertes Dependabot. Notez ce qui manque et corrigez-le.

Solutions des exercices

Solution 1 — CLI de bout en bout

Après installation de gh et gh auth login :

```
gh issue create --title "Ajouter un fichier LICENSE" --body "Le dépôt n'a pas de licence."
# → crée l'issue, affiche son numéro (par ex. #3)

git switch -c feature/license
echo "MIT License ..." > LICENSE
git add . && git commit -m "docs: ajoute une licence MIT (Fixes #3)"
git push -u origin feature/license

gh pr create --fill                # titre/description repris du commit
gh pr checks                       # attendre le vert de la CI (si configurée)
gh pr merge --squash --delete-branch
```

Tout le cycle issue → branche → PR → fusion s'est fait au terminal. Fixes #3 dans le commit ferme l'issue à la fusion (cours 04).

Solution 2 — GitHub Pages

```
echo "<h1>Gestionnaire d'ordres</h1><p>Petit projet de la formation.</p>" >
index.html
git add index.html && git commit -m "docs: page d'accueil GitHub Pages"
git push
```

Sur GitHub : *Settings* → *Pages* → *Build and deployment* → *Source : Deploy from a branch*, choisissez main et le dossier / (root), **Save**. Après une minute, l'URL <https://VOTRE-PSEUDO.github.io/VOTRE-DEPOT/> sert votre page. (Pour un site dans un sous-dossier, choisissez /docs et placez-y index.html.)

Solution 3 — Dependabot

```
mkdir -p .github
cat > .github/dependabot.yml <<'YAML'
version: 2
updates:
  - package-ecosystem: pip
    directory: "/"
    schedule:
      interval: weekly
YAML
```

```
echo "requests==2.20.0" > requirements.txt # version volontairement ancienne
git add . && git commit -m "chore: active Dependabot + dépendance à mettre à jour"
git push
```

Dans l'onglet *Security* → *Dependabot* (ou *Pull requests* peu après), Dependabot ouvre une PR proposant de passer requests à une version récente. On peut aussi forcer une vérification immédiate via *Insights* → *Dependency graph* → *Dependabot* → *Check for updates*.

Solution 4 — Audit de sécurité

Passez en revue, dans *Settings* et *Security* du dépôt :

- **Protection de main** (*Settings* → *Branches*) : PR obligatoire + 1 revue + CI verte (cours 04). Absent ? Ajoutez la règle.
- **Secret scanning** (*Settings* → *Code security*) : activé ? Il alerte si un jeton est committé.

- **Dependabot alerts** (*Security* → *Dependabot*) : des vulnérabilités connues sont-elles listées ?
Traitez-les via les PR proposées.

Notez chaque case décochée et corrigez : c'est exactement le réflexe attendu d'un dépôt public sérieux — et une bonne carte de visite pour un recruteur qui regarde votre GitHub.

Points clés à retenir

- gh couvre l'essentiel des allers-retours vers l'interface web — retenez `pr create`, `pr checks`, `pr merge`, `run watch`.
- **GitHub Pages** publie un site statique gratuitement, en un commit.
- **Dependabot + CI** = des dépendances à jour, sans effort et sans régression.
- La sécurité d'un dépôt se configure **une fois** (protection, scanning, alertes) puis travaille en fond.

Fin de la formation. Parcours accompli : fondements de Git → branches → dépôts distants → collaboration par pull requests → maîtrise de l'historique → intégration continue → bonnes pratiques → écosystème. Vous avez de quoi versionner, partager et automatiser un projet de bout en bout.